Yemekhane Otomasyonu – Dönem Ödevi Raporu

Ders: Web Teknolojileri **Öğrenci:** Mehmet Sefa **Tarih:** Ekim – Kasım 2026 **Git Etiketleri:** `asama-1` `asama-2` `asama-3` `asama-4`

Aşama 2 Mimari Seçimi (İlk Sayfa Beyanı):
Bu projede iki mimari yoldan **2. Yol: Web API + Arayüz** mimarisi seçilmiştir.

- **Arka Uç (Web API):** `ASP.NET Core 8.0 (C#)` ile katmanlı mimaride geliştirilmiş olup yalnızca JSON döner. Veri erişimi `Repositories/YemekhaneRepository.cs` ve `Data/DbConnectionFactory.cs` katmanlarında izole edilmiştir; Controller sınıflarında hiçbir SQL sorgusu bulunmaz.
- **Ön Yüz (Arayüz):** `frontend` container'ı içindeki **Apache 2.4** tarafından sunulan duyarlı (responsive, 400 px telefon uyumlu) `HTML5 + CSS3` ve saf (Vanilla) `JavaScript fetch` ile geliştirilmiştir; React/Vue gibi bir çerçeve kullanılmamıştır. Apache, `/api/*` isteklerini ters vekil (Reverse Proxy) olarak `http://uygulama:8080/api/*` servisine iletir.

Aşama 1 · Altyapı ve MySQL Kaynak-Replika Doğrulaması

Sistem 6 container'dan oluşur: `frontend` (Apache 2.4, port 8080), `uygulama` (ASP.NET Core 8.0 Web API), `mysql-primary` (yazma sunucusu, port 3307), `mysql-replica` (salt okunur okuma sunucusu, port 3308), `replica-setup` (GTID replikasyonunu bir kez kurup `exit 0` ile çıkan kurulum servisi) ve `redis` (önbellek ve oturum deposu, port 6379).

1.1. Tüm Servislerin Healthy Durumu ve replica-setup Çıkışı

```
$ docker compose up -d
$ docker compose ps -a
NAME            IMAGE                          COMMAND                  SERVICE         STATUS                     PORTS
frontend        httpd:2.4-alpine               "httpd-foreground -c…"   frontend        Up 30 minutes (healthy)    0.0.0.0:80
80->80/tcp
mysql-primary   mysql:8.0                      "docker-entrypoint.s…"   mysql-primary   Up 38 minutes (healthy)    0.0.0.0:33
07->3306/tcp
mysql-replica   mysql:8.0                      "docker-entrypoint.s…"   mysql-replica   Up 29 minutes (healthy)    0.0.0.0:33
08->3306/tcp
redis           redis:7-alpine                 "docker-entrypoint.s…"   redis           Up 38 minutes (healthy)    0.0.0.0:63
79->6379/tcp
replica-setup   mysql:8.0                      "/bin/bash /replica-…"   replica-setup   Exited (0) 8 seconds ago
uygulama        yemekhaneotomasyonu-uygulama   "dotnet YemekhaneApi…"   uygulama        Up 8 seconds (healthy)     8080/tcp
```

1.2. Kaynakta Yapılan Bakiye Yüklemesinin Replikada Görünmesi

```
# 1) Öncesi (mysql-replica üzerinde Kullanıcı #1 Ahmet Yılmaz bakiyesi: 750.00 TL)
$ docker exec mysql-replica mysql -uroot -proot_pass_2026 yemekhane -t -e "SELECT id, ad_soyad, eposta, bakiye FROM kullanici
lar WHERE id = 1;"
+----+---------------+---------------------------+--------+
| id | ad_soyad      | eposta                    | bakiye |
+----+---------------+---------------------------+--------+
|  1 | Ahmet Yılmaz  | ahmet.yilmaz@kampus.edu.tr | 750.00 |
+----+---------------+---------------------------+--------+

# 2) Kaynakta (mysql-primary) sp_bakiye_yukle ile 100.00 TL bakiye yüklenmesi:
$ docker exec mysql-primary mysql -uroot -proot_pass_2026 yemekhane -t -e "CALL sp_bakiye_yukle(1, 100.00, 'Aşama 1 replikasy
on testi');"
+--------------+---------------+
| kullanici_id | guncel_bakiye |
+--------------+---------------+
|            1 |        850.00 |
+--------------+---------------+

# 3) Sonrası (mysql-replica üzerinde bakiye GTID replikasyonuyla 850.00 TL olarak görünüyor):
$ docker exec mysql-replica mysql -uroot -proot_pass_2026 yemekhane -t -e "SELECT id, ad_soyad, eposta, bakiye FROM kullanici
lar WHERE id = 1;"
+----+---------------+---------------------------+--------+
| id | ad_soyad      | eposta                    | bakiye |
+----+---------------+---------------------------+--------+
|  1 | Ahmet Yılmaz  | ahmet.yilmaz@kampus.edu.tr | 850.00 |
+----+---------------+---------------------------+--------+
```

1.3. Replikaya Yazma Denemesinin Read-Only Hatasıyla Reddedilmesi

```
$ docker exec mysql-replica mysql -uroot -proot_pass_2026 yemekhane -t -e "UPDATE kullanicilar SET bakiye = 9999.00 WHERE id
= 1;"
ERROR 1290 (HY000) at line 1: The MySQL server is running with the --super-read-only option so it cannot execute this stateme
nt
```

1.4. Replika Durdurulup Yeniden Başlatıldığında Aradaki Değişiklikleri Yakalaması

```
$ docker compose stop mysql-replica
 ✔ Container mysql-replica Stopped
$ docker exec mysql-primary mysql -uroot -proot_pass_2026 yemekhane -t -e "CALL sp_bakiye_yukle(2, 50.00, 'Replika kapalıyken
yükleme');"
+--------------+---------------+
| kullanici_id | guncel_bakiye |
+--------------+---------------+
|            2 |        800.00 |
+--------------+---------------+
$ docker compose start mysql-replica
$ docker exec mysql-replica mysql -uroot -proot_pass_2026 yemekhane -t -e "SELECT id, ad_soyad, eposta, bakiye FROM kullanici
lar WHERE id = 2;"
+----+-------------+--------------------------+--------+
| id | ad_soyad    | eposta                   | bakiye |
+----+-------------+--------------------------+--------+
|  2 | Zeynep Kaya | zeynep.kaya@kampus.edu.tr | 800.00 |
+----+-------------+--------------------------+--------+
```

Aşama 2 · Uygulama Katmanları, İstek Yolu ve Okuma/Yazma Ayrımı

2.1. Bir İsteğin Uygulama İçindeki Yolu (Route → Controller → Veri Katmanı → Yanıt)




```
// 1) Controller Katmanı: uygulama/Controllers/RezervasyonController.cs (İçinde SQL bulunmaz)
[Route("api/rezervasyonlar")]
public class RezervasyonController : BaseApiController
{
    [HttpPost]
    public async Task<IActionResult> Olustur([FromBody] RezervasyonOlusturDto istek)
    {
        var oturum = await AktifOturumGetirAsync(); // Redis 'oturum:{token}' doğrulaması + 30 dk TTL yenileme
        if (oturum == null) return Unauthorized(new { hata = "Rezervasyon yapmak için lütfen giriş yapın." });

        try {
            var (rezId, kesilenUcret, guncelBakiye, porsiyon) =
                await _repository.RezervasyonYapAsync(oturum.Id, istek.MenuId, istek.Porsiyon);

            oturum.Bakiye = guncelBakiye;
            await RedisService.OturumGuncelleAsync(IstekTokenGetir()!, oturum);

            return Ok(new {
                mesaj = "Rezervasyonunuz başarıyla oluşturuldu.",
                rezervasyonId = rezId, porsiyon, kesilenUcret, guncelBakiye,
                bakiyeOkumaKaynagi = "mysql-primary"
            });
        }
        catch (IsKuraliIstisnasi ex) {
            // Stored Procedure SQLSTATE '45000' Türkçe hata mesajı HTTP 400 ile arayüze iletilir
            return BadRequest(new { hata = ex.Message });
        }
    }
}

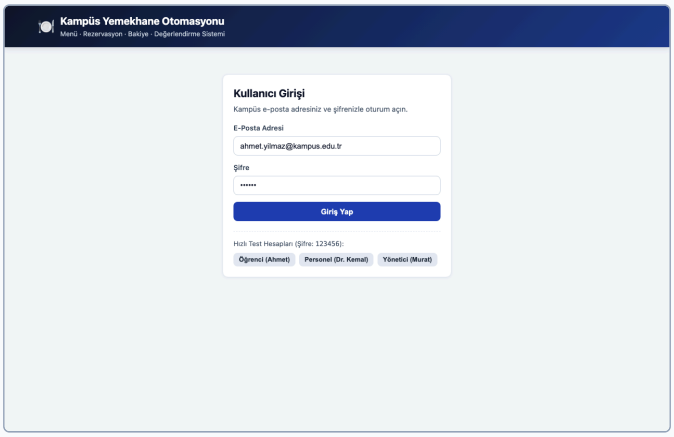
// 2) Veri Erişim Katmanı: uygulama/Repositories/YemekhaneRepository.cs
public async Task<(int RezervasyonId, decimal KesilenUcret, decimal GuncelBakiye, string Porsiyon)>
    RezervasyonYapAsync(int kullaniciId, int menuId, string porsiyon = "tam")
{
    await using var conn = await _dbFactory.OpenPrimaryConnectionAsync(); // Yazma -> mysql-primary:3306
    try {
        await using var cmd = new MySqlCommand("sp_rezervasyon_yap", conn) { CommandType = CommandType.StoredProcedure };
        cmd.Parameters.AddWithValue("@p_kullanici_id", kullaniciId);
        cmd.Parameters.AddWithValue("@p_menu_id", menuId);
        cmd.Parameters.AddWithValue("@p_porsiyon", porsiyon);
        // ... Stored Procedure çalıştırılır ve hemen ardından bakiye mysql-primary'den okunur ...
    }
    catch (MySqlException ex) when (ex.SqlState == "45000") {
        throw new IsKuraliIstisnasi(ex.Message); // Örn: "Bakiye yetersiz." veya "Kapasite dolmuştur."
    }
}
```

2.2. Rezervasyondan Hemen Sonra Gösterilen Bakiyenin Neden Kaynaktan (mysql-primary) Okunduğu

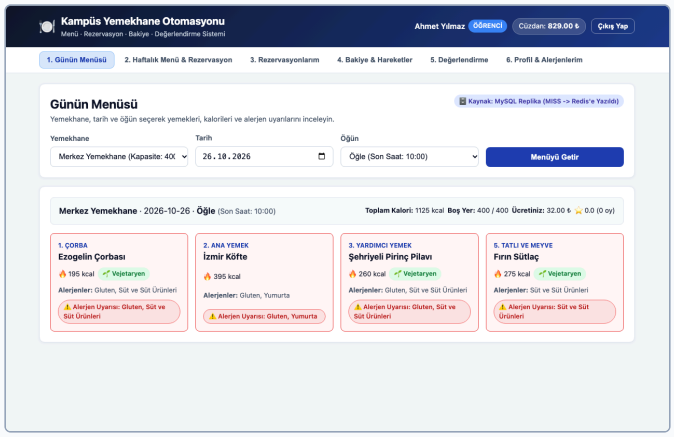
Replikasyon Gecikmesi (Replication Lag) ve Read-Your-Own-Writes Tutarlılığı:
MySQL kaynak-replika (source-replica) mimarisinde replikasyon varsayılan olarak **asenkron (eşzamansız)** çalışır. Kullanıcı rezervasyon yaptığında `sp_rezervasyon_yap` yordamı `mysql-primary` üzerinde kullanıcının bakiyesini düşürür ve işlemi `COMMIT` eder; ancak bu işlemin binary log (GTID) üzerinden `mysql-replica` sunucusuna iletilip relay log'dan uygulanması ağ veya disk yüküne bağlı olarak birkaç milisaniye ile birkaç saniye arasında gecikebilir (*replikasyon gecikmesi / replication lag*). Eğer rezervasyon işlemi tamamlanır tamamlanmaz kullanıcının yeni bakiyesi `mysql-replica` üzerinden okunsaydı, replika henüz güncellenmemiş olabileceği için kullanıcı ekranda eski (düşülmemiş) bakiyesini görebilirdi. Bu tutarsızlığı (*stale read*) önlemek ve **Read-Your-Own-Writes** tutarlılığını garanti altına almak adına, rezervasyon ve bakiye yükleme işlemlerinin hemen ardından dönen güncel bakiye doğrudan yazma işleminin yapıldığı `mysql-primary` sunucusundan okunmaktadır.

2.3. Arayüz Ekran Görüntüleri (7 Zorunlu Ekranın Tamamı)

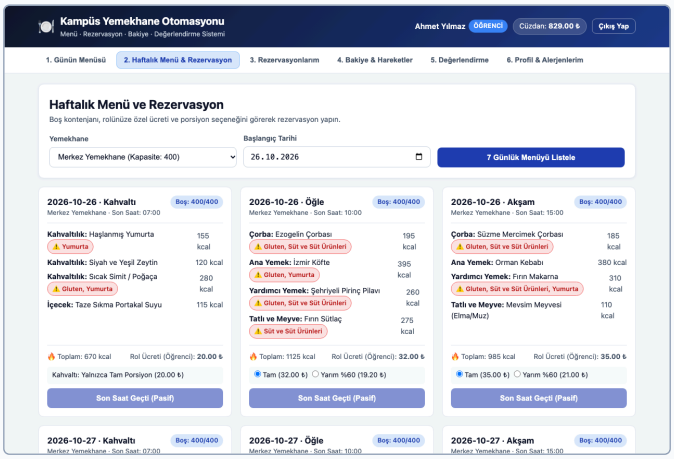
Ekran 1: Giriş ve Çıkış (E-posta + Şifre, Bcrypt)



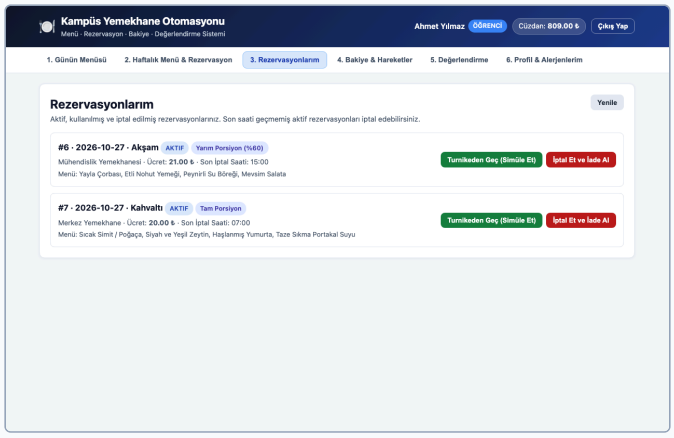
Ekran 2: Günün Menüsü (Kalori, Vejetaryen, Alerjen Uyarısı)



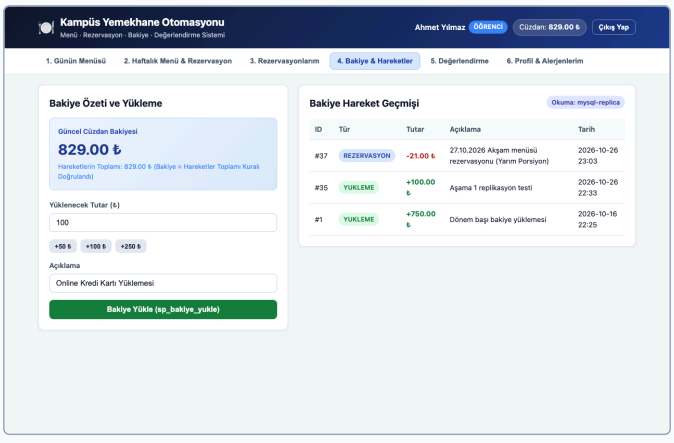
Ekran 3: Haftalık Menü ve Rezervasyon (Tam/Yarım Porsiyon)



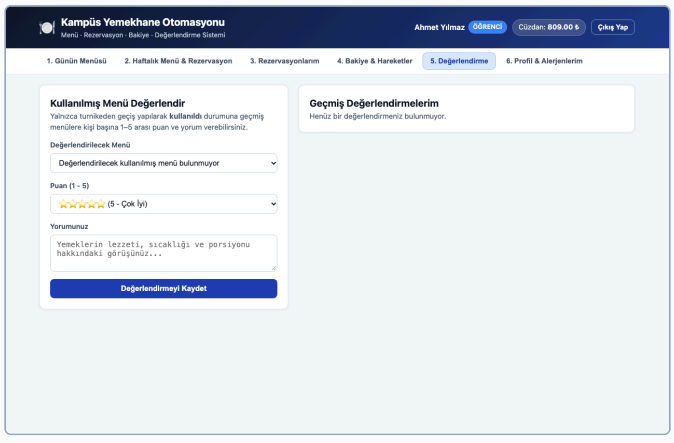
Ekran 4: Rezervasyonlarım (Aktif, Kullanıldı, İptal)



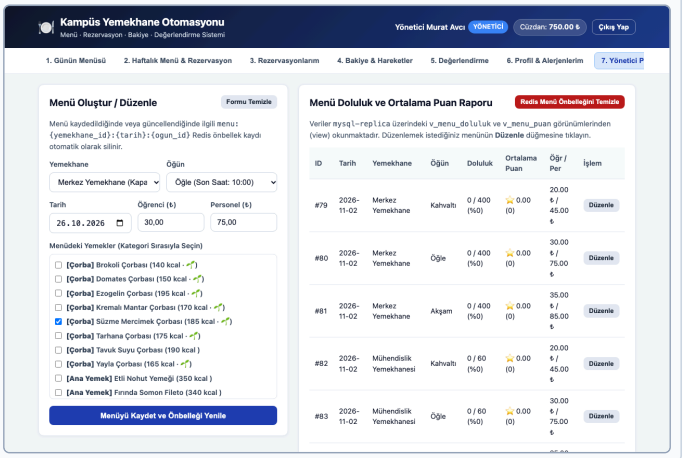
Ekran 5: Bakiye (Güncel Bakiye, Hareketler, Bakiye Yükleme)



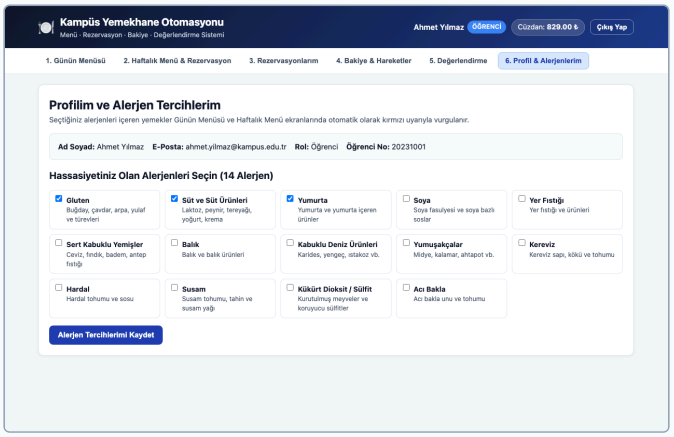
Ekran 6: Değerlendirme (Kullanılmış Menüye 1–5 Puan ve Yorum)



Ekran 7: Yönetici Paneli (Menü Oluştur/Düzenle, Doluluk ve Puan)



Aşama 4 Ekranı: Profilim ve 14 Alerjen Seçimi



Aşama 3 · Redis Önbellek ve Oturum Deposu Doğrulaması

3.1. Oturum ve Menü Önbelleği redis-cli Çıktıları

```
# 1) Kullanıcı girişi sonrası Redis'te oluşturulan oturum anahtarı ve 30 dk (1800 sn) TTL süresi:
$ docker exec redis redis-cli KEYS "oturum:*"
oturum:0358149df97d406a8d810ddfccfb2d3b
$ docker exec redis redis-cli TTL "oturum:0358149df97d406a8d810ddfccfb2d3b"
1800
$ docker exec redis redis-cli GET "oturum:0358149df97d406a8d810ddfccfb2d3b"
{"Id":1,"AdSoyad":"Ahmet Y\u0131lmaz","Eposta":"ahmet.yilmaz@kampus.edu.tr","Rol":"ogrenci","OgrenciNo":"20231001","Bakiye":8
50.00,"AlerjenIdleri":[1,2]}

# 2) Günün menüsü ilk istekte DB'den okunup gün sonuna kadar Redis'e yazılır:
$ docker exec redis redis-cli KEYS "menu:*"
menu:1:2026-10-26:2
$ docker exec redis redis-cli TTL "menu:1:2026-10-26:2"
3364
```

3.2. Uygulama Günlüğü (İlk İstek DB, İkincisi Redis ve Menü Güncellemede Invalidation)

```
$ docker logs uygulama
[REDIS SESSION CREATE] Oturum oluşturuldu: oturum:0358149df97d406a8d810ddfccfb2d3b (Kullanıcı: ahmet.yilmaz@kampus.edu.tr, TT
L: 1800 sn)
[REDIS CACHE MISS] İlk istek: 'menu:1:2026-10-26:2' Redis'te bulunamadı, veritabanından (mysql-replica) okunuyor.
[DB READ - REPLICA] v_menu_doluluk, v_menu_puan ve v_menu_detay sorgulanıyor (Yemekhane:1, Tarih:2026-10-26, Öğün:2)
[REDIS CACHE SET] Menü Redis önbelleğine yazıldı: 'menu:1:2026-10-26:2' (TTL: 3364 sn, gün sonuna kadar)
[REDIS CACHE HIT] İstek Redis önbelleğinden karşılandı: 'menu:1:2026-10-26:2'
[DB WRITE - PRIMARY] Menü kaydedildi/güncellendi (MenuId:38, Yemekhane:1, Tarih:2026-10-26, Öğün:2)
[REDIS CACHE INVALIDATE] Menü güncellendiği için önbellek kaydı silindi: 'menu:1:2026-10-26:2' (Silindi: True)
```

3.3. Uygulama Container'ı Yeniden Başlatıldığında Oturumun Korunması

```
$ docker compose restart uygulama
 ✔ Container uygulama Started
$ curl -sS http://localhost:8080/api/auth/me -H "Authorization: Bearer 0358149df97d406a8d810ddfccfb2d3b"
{"id":1,"adSoyad":"Ahmet Yılmaz","eposta":"ahmet.yilmaz@kampus.edu.tr","rol":"ogrenci","ogrenciNo":"20231001","bakiye":850.0
0,"alerjenIdleri":[1,2]}
```

Aşama 4 · Değişiklik Talebi ve Migration (04_degisiklik.sql)

4.1. Migration Uygulaması ve Replikaya Geçtiğinin Kanıtı (SHOW CREATE TABLE)

```
$ docker exec -i mysql-primary mysql -uroot -proot_pass_2026 yemekhane < 04_degisiklik.sql
$ docker exec mysql-replica mysql -uroot -proot_pass_2026 yemekhane -e "SHOW CREATE TABLE rezervasyonlar\G"
*************************** 1. row ***************************
       Table: rezervasyonlar
Create Table: CREATE TABLE `rezervasyonlar` (
  `id` int NOT NULL AUTO_INCREMENT,
  `kullanici_id` int NOT NULL,
  `menu_id` int NOT NULL,
  `durum` enum('aktif','kullanildi','iptal') COLLATE utf8mb4_turkish_ci NOT NULL DEFAULT 'aktif',
  `porsiyon` enum('tam','yarim') COLLATE utf8mb4_turkish_ci NOT NULL DEFAULT 'tam',
  `ucret` decimal(10,2) NOT NULL,
  `olusturulma_tarihi` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `guncellenme_tarihi` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_kullanici_menu` (`kullanici_id`,`menu_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_turkish_ci

$ docker exec mysql-replica mysql -uroot -proot_pass_2026 yemekhane -e "SHOW CREATE TABLE kullanici_alerjenleri\G"
*************************** 1. row ***************************
       Table: kullanici_alerjenleri
Create Table: CREATE TABLE `kullanici_alerjenleri` (
  `kullanici_id` int NOT NULL,
  `alerjen_id` int NOT NULL,
  `olusturulma_tarihi` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`kullanici_id`,`alerjen_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_turkish_ci
```

4.2. Eski Redis Menü Kayıtlarının Temizlenmesi ve Yarım Porsiyon / SQLSTATE 45000 Testleri

```
# 1) Eski yapıdaki Redis menü kayıtlarının temizlenmesi:
$ curl -sS -X POST http://localhost:8080/api/yonetici/cache-temizle -H "Authorization: Bearer $ADMIN_TOKEN"
{"mesaj":"Redis üzerindeki 1 adet menü önbellek kaydı temizlendi.","silinenSayisi":1}

# 2) Kahvaltı menüsüne yarım porsiyon denemesi (sp_rezervasyon_yap SQLSTATE 45000 Türkçe hata döndürür):
$ curl -sS -X POST http://localhost:8080/api/rezervasyonlar -H "Authorization: Bearer $TOKEN" -d '{"menuId":43,"porsiyon":"ya
rim"}'
{"hata":"Kahvaltı öğününde yarım porsiyon seçeneği bulunmamaktadır."}

# 3) Akşam menüsüne (Tam fiyat: 35.00 TL) yarım porsiyon (%60 = 21.00 TL) rezervasyonu:
$ curl -sS -X POST http://localhost:8080/api/rezervasyonlar -H "Authorization: Bearer $TOKEN" -d '{"menuId":48,"porsiyon":"ya
rim"}'
{"mesaj":"Rezervasyonunuz başarıyla oluşturuldu.","rezervasyonId":6,"porsiyon":"yarim","kesilenUcret":21.00,"guncelBakiye":82
9.00,"bakiyeOkumaKaynagi":"mysql-primary"}
```

4.3. Telefon Genişliği (~400 px Duyarlı Mobil Görünüm)

400 px Telefon Genişliği Görünümü

